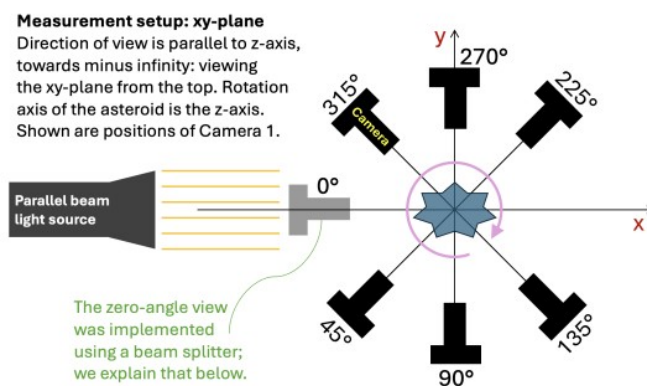


實驗設計簡介

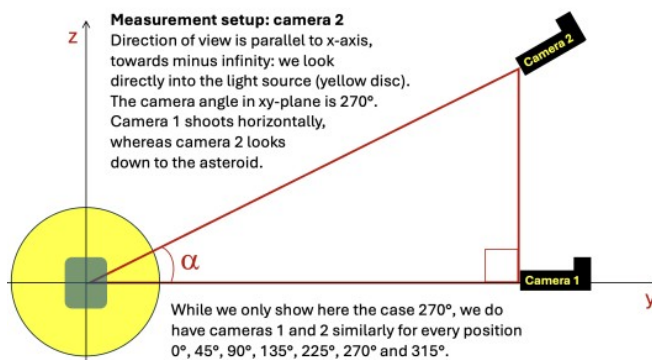
這裏我只簡單介紹設定，細節矯正這裏就不多說，因為我們也沒打算重現他們的實驗。
這次比賽的設計用到的物品只有三樣東西：（發出平行光源的）手電筒以及兩臺一樣的相機。

我們先看看第一臺相機如何設置：



圖一

簡單來說，手電筒以及相機都方在桌面上，可以簡單理解為 xy -axis。第二臺相機就直接架設在第一臺相機的正上方，如下圖：



圖二

固定好（發出平行光源的）手電筒以及兩臺一樣的相機後，他們便把“隕石”方在光源前面，旋轉錄影。到目前為止，主辦方其實做的事情就是：

step 1: 先用電腦畫 stl 檔案

step 2: 用 3D 打印機輸出“隕石”

step 3: 按照上述設定錄製 mp4 檔案（grayscale），再根據 mp4 檔案畫 (binary/intensity) light curve。

資料簡介

對於每一個“隕石”，他們只做了 7 次測量：

	放在桌上相機（我們稱之為“第一臺相機”）的位置，如圖一	“第二臺相機”（桌上相機的正上方）的位置，如圖二	Light curve 編碼對應的位置
第 1 次測量	0 度	$\alpha = 21$ 度	Columns 2, 3, 4, 5
第 2 次測量	45 度	$\alpha = 26$ 度	Columns 6, 7, 8, 9
第 3 次測量	90 度	$\alpha = 26$ 度	Columns 10, 11, 12, 13
第 4 次測量	135 度	$\alpha = 26$ 度	Columns 14, 15, 16, 17
第 5 次測量	225 度	$\alpha = 24$ 度	Columns 18, 19, 20, 21
第 6 次測量	270 度	$\alpha = 24$ 度	Columns 22, 23, 24, 25
第 7 次測量	315 度	$\alpha = 24$ 度	Columns 26, 27, 28, 29

表一

Column 1 是幀數/時間（也就是 360 度的均勻分割數）。根據 Asteroid01_lightcurve_intensity.txt，他們共做了 841 個均勻分割。

更準確的來說，
Column 2 是第一次測量，“第一臺相機”所繪製的 light intensity curve。這個 column 是“真實資料”。
Column 3 是第一次測量，計算模擬“第一臺相機”所繪製的 light binary curve。這個 column 是“模擬資料”。
Column 4 是第一次測量，“第二臺相機”所繪製的 light intensity curve。這個 column 是“真實資料”。
Column 5 是第一次測量，計算模擬“第二臺相機”所繪製的 light binary curve。這個 column 是“模擬資料”。

類似的，
Column 6 是第二次測量，“第一臺相機”所繪製的 light intensity curve。這個 column 是“真實資料”。
Column 7 是第二次測量，計算模擬“第一臺相機”所繪製的 light binary curve。這個 column 是“模擬資料”。
Column 8 是第二次測量，“第二臺相機”所繪製的 light intensity curve。這個 column 是“真實資料”。
Column 9 是第二次測量，計算模擬“第二臺相機”所繪製的 light binary curve。這個 column 是“模擬資料”。

剩下的就以此類推，我就不一一列舉。

訓練 CNN 網路

首先，我們要先瞭解 stl 檔案。我直接用 TextEditor (就是用來開 txt 檔案) 來打開，裏面意外的整齊。以 asteroid1.stl 為例，裏面是這樣的：

```
solid Created by Gmsh
facet normal -0.159289 0.167131 0.972982
  outer loop
    vertex -1.888296041844072 0.0002460429996352797 3.161097003616941
    vertex -1.890329489593098 0.01126283538653845 3.158871721174612
    vertex -1.90946363000239 0.0006845222986165005 3.157556283277667
  endloop
endfacet
facet normal 0.0283625 -0.328219 0.944176
  outer loop
    vertex -0.174033817520803 -1.862797688459199 3.159907628518454
    vertex -0.1963030849178117 -1.864990084954105 3.159814451667421
    vertex -0.1809672714359435 -1.879416053890588 3.154338941421392
  endloop
endfacet
facet normal -0.460625 -0.794076 0.39657
  outer loop
    vertex -2.445548420936831 -3.041386196190822 0.7772878517318224
    vertex -2.430985427219417 -3.046417746146632 0.7841281287959293
    vertex -2.445625154814153 -3.036135406585522 0.7877126970651008
  endloop
endfacet
(以下略)
```

理論上，第二臺相機的位置是應該要 labeling 的，但我發現到 7 次測量其實都差不多，在資料訓練時其實不必做 labeling。

因為 Column 3,5,7, ..., 29 其實不是真實資料，且我有人工比對過其實和真實資料差不多，所以其實可以直接拿掉。還句話來說，其實 Column 3,5,7, ..., 29 沒有提供任何有效的新信息，倒是給出不必要的噪音。這些 column 都要刪掉，所以最後會剩下 15 個 column。因為主辦方對角度做了 841，所以就是 15×841 的矩陣。

從資料開看，訓練 CNN 網絡時馬上就遇到了第一個難題：我們不知道需要多少個三角形來逼近曲面，這樣會造成資料維度無法統一的問題。

為了解決這個問題，我們必須還個方式對立體進行編碼。我們使用 voxel (**v**olume **p**ixel) 的概念。目前我看過比賽規則，所有的“隕石”都被限制在 $[-2, 2] \times [-2, 2] \times [-1, 1]$ 的長方形內。考慮到儲存所需容量，目前看起來比較可行的方法是用邊長為 0.001 的小正方形組成，這樣就是 $400 \times 400 \times 200$ 的矩陣，裏面只能是 0（無）或 1（有）。根據 ChatGPT 的估計，如果用 bool 或 uint8 來儲存，大約就 32MB，所以是可行的。下列是 ChatGPT 建議的程式，需要你幫忙 debug：

HAC2026 筆記（普照的個人觀點）

```
import numpy as np
import trimesh

mesh = trimesh.load_mesh("object.stl")
vox = mesh.voxelized(pitch=0.01).fill()
V = np.zeros((400, 400, 200), dtype=np.uint8)
points = vox.points

x = points[:, 0]
y = points[:, 1]
z = points[:, 2]

# -----
# Convert coordinates to tensor indices
# -----
i = np.floor((x + 2.0) / 0.01).astype(int)
j = np.floor((y + 2.0) / 0.01).astype(int)
k = np.floor((z + 1.0) / 0.01).astype(int)

# Keep only voxels inside the box
mask = (
    (0 <= i) & (i < 400) &
    (0 <= j) & (j < 400) &
    (0 <= k) & (k < 200)
)

i = i[mask]
j = j[mask]
k = k[mask]

# -----
# Fill occupancy tensor
# -----
V[i, j, k] = 1

print("Tensor shape:", V.shape)
print("Occupied voxels:", V.sum())

np.save("object_voxelized.npy", V)
V = np.load("object_voxelized.npy")
```

Load STL
Voxelize and fill interior
Global tensor
Physical coordinates of occupied voxels

Save to disk
Load later

HAC2026 筆記（普照的個人觀點）

下一個問題就是如何從 .npy 返回 .stl 。下列為 ChatGPT 給的建議，需要你幫忙 debug：

```
import numpy as np
import trimesh
from skimage.measure import marching_cubes

V = np.load("object_voxelized.npy") # Load voxel tensor

# -----
# Extract surface
# -----
verts, faces, normals, values = marching_cubes(
    V,
    level=0.5
)

# -----
# Convert voxel indices back to physical coordinates
# -----
dx = dy = dz = 0.01

verts[:, 0] = -2.0 + verts[:, 0] * dx
verts[:, 1] = -2.0 + verts[:, 1] * dy
verts[:, 2] = -1.0 + verts[:, 2] * dz

mesh = trimesh.Trimesh( # Create mesh
    vertices=verts,
    faces=faces
)

mesh.export("reconstructed.stl") # Save STL
```

不過這樣會遇到一個問題：機器學習可能生成出來的東西，裏面會出現洞，或者外面出現“顆粒”。我們只需要保留最大的 3D 連通物件。下列為 ChatGPT 給的建議，需要你幫忙 debug：

```
from scipy.ndimage import label

labels, n = label(V)

sizes = np.bincount(labels.ravel())
largest = sizes[1:].argmax() + 1

V_clean = (labels == largest)
```

逆向工程（後續再做）

5月30日更新：我後來發現下列事情完全可以用 Blender 實現。Blender （<https://www.blender.org/>）是一套免費且開放原始碼的 3D 電腦繪圖軟體。它具備強大的多功能性，涵蓋了 3D 建模、動畫製作、材質貼圖、燈光渲染、視覺特效（如流體與煙霧）、影片剪輯到 3D 列印支援等完整流程，完全適用於商業專案。有需要的可以從 <https://damit.cuni.cz/projects/damit/asteroids/browse> 下載大量真實隕石的 obj 檔案。

5月29日的筆記：
我們現在要對他們的模擬做逆向工程（reverse engineering），俗稱破解。
因為他們沒有提供 Blender 如何計算的，所以 Blender 所模擬的資料沒有任何意義，其實是個煙霧彈。
很顯然，我們的破解一定是根據真實資料來比對。

SSS

裏面原來很貼心的給了 normal，是指向外面的（還很貼心的單位化了，超讚的！），所以連這個都不用計算了。那麼事情就簡單了，我打算根據下列文章來設計：
Kaasalainen, Mikko, and Johanna Torppa. “Optimization methods for asteroid lightcurve inversion: I. shape determination.” *Icarus* 153.1 (2001): 24-36.

首先，主辦方的“隕石”是用 3D 打印機列印的，也就是說是一材料。這樣反照率（albedo） ϖ 基本上可以假設是常數。因為“隕石”的反照率相對低，所以可以忽略二次反射，因此 lightcurve 大致上可以估計成

$$L = \varpi \sum_{\Delta \in \{\text{facet}\}} S(\mu, \mu_0) \text{area}(\Delta)$$

其中 $\mu = \mathbf{E} \cdot \mathbf{n}_\Delta$ 以及 $\mu_0 = \mathbf{E}_0 \cdot \mathbf{n}_\Delta$ ，其中 $\mathbf{n}_\Delta$ 是剛剛提到的 outward unit normal，stl 檔案很貼心的提供了。這裏我們就簡單使用 Lambert law $S(\mu, \mu_0) = (\mu\mu_0)_+ = \max\{\mu\mu_0, 0\}$ 就好。當然，以後再看看 Lommel-Seeliger law，我想這次比賽應該不會那麼複雜。

這裏 $\mathbf{E}_0$ 為向着光源的方向，以及 $\mathbf{E}$ 為向着觀測者的方向。這兩個向量都和 Δ 無關。因為光源放在桌上所以 $\mathbf{E}_0 = (E_{0,1}, E_{0,2}, 0)$ 。不失一般性，我們矯正為 $\mathbf{E}_0 = (-1, 0, 0)$ ，也就是如圖一的設定。

下列為表一”第一臺相機“的換算：

	放在桌上相機（我們稱之為“第一臺相機”）的位置，如圖一
第 1 次測量	$\mathbf{E} = \mathbf{E}_0$
第 2 次測量	$\mathbf{E} = \begin{pmatrix} \cos(\pi/4) & -\sin(\pi/4) & 0 \\ \sin(\pi/4) & \cos(\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \mathbf{E}_0$
第 3 次測量	$\mathbf{E} = \begin{pmatrix} \cos(2\pi/4) & -\sin(2\pi/4) & 0 \\ \sin(2\pi/4) & \cos(2\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \mathbf{E}_0$

第 4 次測量	$\mathbf{E} = \begin{pmatrix} \cos(3\pi/4) & -\sin(3\pi/4) & 0 \\ \sin(3\pi/4) & \cos(3\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \mathbf{E}_0$
第 5 次測量	$\mathbf{E} = \begin{pmatrix} \cos(5\pi/4) & -\sin(5\pi/4) & 0 \\ \sin(5\pi/4) & \cos(5\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \mathbf{E}_0$
第 6 次測量	$\mathbf{E} = \begin{pmatrix} \cos(6\pi/4) & -\sin(6\pi/4) & 0 \\ \sin(6\pi/4) & \cos(6\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \mathbf{E}_0$
第 7 次測量	$\mathbf{E} = \begin{pmatrix} \cos(7\pi/4) & -\sin(7\pi/4) & 0 \\ \sin(7\pi/4) & \cos(7\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \mathbf{E}_0$

表二

下列為表一” 第二臺相機 “的換算：

	“第二臺相機”（桌上相機的正上方）的位置，如圖二		
第 1 次測量	$\mathbf{E} = \begin{pmatrix} \cos(21\pi/180) & 0 & \sin(21\pi/180) \\ 0 & 1 & 0 \\ -\sin(21\pi/180) & 0 & \cos(21\pi/180) \end{pmatrix} \mathbf{E}_0$		
第 2 次測量	$\mathbf{E} = \begin{pmatrix} \cos(\pi/4) & -\sin(\pi/4) & 0 \\ \sin(\pi/4) & \cos(\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos(26\pi/180) & 0 & \sin(26\pi/180) \\ 0 & 1 & 0 \\ -\sin(26\pi/180) & 0 & \cos(26\pi/180) \end{pmatrix} \mathbf{E}_0$		
第 3 次測量	$\mathbf{E} = \begin{pmatrix} \cos(2\pi/4) & -\sin(2\pi/4) & 0 \\ \sin(2\pi/4) & \cos(2\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos(26\pi/180) & 0 & \sin(26\pi/180) \\ 0 & 1 & 0 \\ -\sin(26\pi/180) & 0 & \cos(26\pi/180) \end{pmatrix} \mathbf{E}_0$		
第 4 次測量	$\mathbf{E} = \begin{pmatrix} \cos(3\pi/4) & -\sin(3\pi/4) & 0 \\ \sin(3\pi/4) & \cos(3\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos(26\pi/180) & 0 & \sin(26\pi/180) \\ 0 & 1 & 0 \\ -\sin(26\pi/180) & 0 & \cos(26\pi/180) \end{pmatrix} \mathbf{E}_0$		
第 5 次測量	$\mathbf{E} = \begin{pmatrix} \cos(5\pi/4) & -\sin(5\pi/4) & 0 \\ \sin(5\pi/4) & \cos(5\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos(24\pi/180) & 0 & \sin(24\pi/180) \\ 0 & 1 & 0 \\ -\sin(24\pi/180) & 0 & \cos(24\pi/180) \end{pmatrix} \mathbf{E}_0$		
第 6 次測量	$\mathbf{E} = \begin{pmatrix} \cos(6\pi/4) & -\sin(6\pi/4) & 0 \\ \sin(6\pi/4) & \cos(6\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos(24\pi/180) & 0 & \sin(24\pi/180) \\ 0 & 1 & 0 \\ -\sin(24\pi/180) & 0 & \cos(24\pi/180) \end{pmatrix} \mathbf{E}_0$		
第 7 次測量	$\mathbf{E} = \begin{pmatrix} \cos(7\pi/4) & -\sin(7\pi/4) & 0 \\ \sin(7\pi/4) & \cos(7\pi/4) & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos(24\pi/180) & 0 & \sin(24\pi/180) \\ 0 & 1 & 0 \\ -\sin(24\pi/180) & 0 & \cos(24\pi/180) \end{pmatrix} \mathbf{E}_0$		

表三

HAC2026 筆記（普照的個人觀點）

假設三角形 Δ 的三個頂點為 $\mathbf{a} = (a_1, a_2, a_3)$, $\mathbf{b} = (b_1, b_2, b_3)$, $\mathbf{c} = (c_1, c_2, c_3)$, 那麼

$$\text{area}(\Delta) = \frac{1}{2} |(\mathbf{a} - \mathbf{b}) \times \mathbf{a} - \mathbf{c}|$$

其中 $\times$ 為 cross product, 見 https://en.wikipedia.org/wiki/Cross_product。stl 檔案都很貼心的幫我們準備好了, 超讚的!

這樣, 7 次測量以及兩臺相機就可以拿到兩條 light curve (做 841 個分割), 加上 Column 1 是幀數/時間 (也就是 360 度的均勻分割數), 就會有 15 個 column (15 $\times$ 841 的矩陣), 這樣就對上了。

這裏要注意一點, 主辦方的 light curve 已經做過 normalization。他們的做法是, 離散後的 lightcurve 取平均, 所以他們提供的其實 $L/\text{average}$ 。